```
    create_table :physicians do |t|
      t.string :name
      t.timestamps null: false
    end

    create_table :patients do |t|
      t.string :name
      t.timestamps null: false
    end

    create_table :appointments do |t|
      t.belongs_to :physician, index: true
      t.belongs_to :patient, index: true
      t.datetime :appointment_date
      t.timestamps null: false
    end
  end
end
```

The collection of join models can be managed via the API. For example, if you assign

```
physician.patients = patients
```

new join models are created for newly associated objects, and if some are gone their rows are deleted.

Automatic deletion of join models is direct, no destroy callbacks are triggered.

The `has_many :through` association is also useful for setting up "shortcuts" through nested `has_many` associations. For example, if a document has many sections, and a section has many paragraphs, you may sometimes want to get a simple collection of all paragraphs in the document. You could set that up this way:

```
class Document < ActiveRecord::Base
  has_many :sections
  has_many :paragraphs, through: :sections
end

class Section < ActiveRecord::Base
  belongs_to :document
  has_many :paragraphs
end

class Paragraph < ActiveRecord::Base
  belongs_to :section
end
```

With `through: :sections` specified, Rails will now understand:

```
@document.paragraphs
```

2.5 The `has_one :through` Association

A `has_one :through` association sets up a one-to-one connection with another model. This association indicates that the declaring model can be matched with one instance of another model by proceeding *through* a third model. For example, if each supplier has one account, and each account is associated with one account history, then the supplier model could look like this:

```
class Supplier < ActiveRecord::Base
  has_one :account
  has_one :account_history, through: :account
end

class Account < ActiveRecord::Base
  belongs_to :supplier
  has_one :account_history
end
```